CO3 – Assessment Tool 3

Divide-and-Conquer Engine of the KartIQ Retail Data Analytics Platform

Student Name	R. Visshaal
Registration No.	192421025
Course	CSA06 – Design and Analysis of Algorithms
Assessment	100 points
Submission date	26 October 2026

Worked reference solution based on the supplied three-page assignment PDF.

Final Design and Analysis — KartIQ

(a) Understand the case

Module • Requirement and key issue • Poor choice consequence • Design

Nightly order consolidation • Sort 10 million orders by timestamp; stability required for equal timestamps; memory may exceed RAM • Unstable ties break gateway arrival order; quadratic sorting misses deadline • Stable Merge Sort; external merge sort if data exceeds RAM

Live seller leaderboard • 100,000 scores; data nearly sorted; first-element pivot creates unbalanced partitions • Near-quadratic comparisons and visible sale-event freezes • Randomized pivot / introspective sort; benchmark Timsort if updates are sparse

Order lookup • Find timestamp in sorted consolidated orders; return first duplicate • Linear scan has O(n) lookup; ordinary binary search may return an arbitrary duplicate • Lower-bound binary search, O(log n)

Recommendation scores • Multiply two 1,024 × 1,024 matrices • Conventional method uses many multiplications; pure Strassen incurs overhead and temporary memory • Hybrid Strassen with conventional base case and measured cutoff

(b) Merge Sort

Sample input:

[(105,O1),(102,O2),(105,O3),(101,O4),(102,O5),(103,O6)]

Split:

- [O1,O2,O3] and [O4,O5,O6]

- [O1] and [O2,O3]; [O4] and [O5,O6]

- [O2] and [O3]; [O5] and [O6]

Merge trace:

1. [O2] + [O3] -> [(102,O2),(105,O3)]

2. [O1] + [(102,O2),(105,O3)] -> [(102,O2),(105,O1),(105,O3)]

3. [O5] + [O6] -> [(102,O5),(103,O6)]

4. [O4] + [(102,O5),(103,O6)] -> [(101,O4),(102,O5),(103,O6)]

5. Final merge -> [(101,O4),(102,O2),(102,O5),(103,O6),(105,O1),(105,O3)]

Stability: the merge chooses from the left run when keys tie (<=). The two timestamp-102 orders retain O2 before O5, and timestamp-105 orders retain O1 before O3.

Recurrence: T(n)=2T(n/2)+Θ(n), T(1)=Θ(1). By the Master Theorem, T(n)=Θ(n log■ n).

For 10,000,000 orders, n log■ n ≈ 10,000,000 × 23.2535 ≈ 232,535,000. This is a scale estimate, not an exact comparison count. A standard merge sort comparison upper bound is below n■log■ n■ – 2^■log■ n■ + 1, approximately 233 million here.

Auxiliary memory: Θ(n), in addition to the records. If RAM is insufficient, sort chunks that fit in memory, write sorted runs to disk, then perform a k-way external merge; buffer sequential disk reads/writes.

(c) Quick Sort

Sample scores: [42,17,63,8,55,29]. With first-element pivot 42 and a Lomuto-style partition variant that places values ≤ pivot on the left, one valid partition result is [17,8,29,42,55,63], with pivot 42 at index 3. Exact intermediate arrangement depends on the partition scheme.

Recurrences:

- Balanced/best: T(n)=2T(n/2)+cn = Θ(n log n).

- Worst (pivot is smallest/largest each time): T(n)=T(n–1)+cn = Θ(n²).

For 100,000 already-nearly-sorted scores, first-element pivot can cause roughly n(n–1)/2 = 4,999,950,000 comparisons in the worst pattern. This is a worst-case estimate, not a prediction for every nearly sorted input.

Improvement: randomized pivot reduces dependence on input order and has expected Θ(n log n) time; median-of-three plus 3-way partitioning helps practical data with repeated scores. Introsort switches to heapsort if recursion depth grows too large.

For only a few changed scores, incremental maintenance with a heap/tree may avoid re-sorting all sellers, but requires careful handling of rank queries.

(d) Binary Search — first occurrence

Binary search requires a sorted array; Merge Sort provides this invariant. At each step, if A[mid] >= target, move high left; when equal, record mid and continue left. This is lower-bound search.

Recurrence: T(n)=T(n/2)+Θ(1), T(1)=Θ(1), giving Θ(log■ n).

For 10,000,000 records, ■log■(10,000,000+1)■ = 24 comparisons as a conservative maximum bound for a binary decision path; actual iterations depend on target and implementation. The algorithm returns the first matching index or -1.

(e) Strassen matrix multiplication

For 2×2 matrices A=[[a,b],[c,d]], B=[[e,f],[g,h]]:

- M1=(a+d)(e+h)
- M2=(c+d)e
- M3=a(f–h)
- M4=d(g–e)
- M5=(a+b)h
- M6=(c–a)(e+f)
- M7=(b–d)(g+h)
- C11=M1+M4–M5+M7
- C12=M3+M5
- C21=M2+M4
- C22=M1–M2+M3+M6

Sample A=[[1,7],[3,5]], B=[[1,3],[7,5]]:

M1=36, M2=8, M3=–2, M4=30, M5=40, M6=8, M7=24.

Then C11=72+30–40+16=78; C12=–2+40=38; C21=8+30=38; C22=72–8–2–16=46.

Thus C=[[78,38],[38,46]], also confirmed by conventional multiplication.

Recurrence: T(n)=7T(n/2)+Θ(n²). Master theorem gives Θ(n^(log■7)) ≈ Θ(n^2.8074).

For n=1024, conventional multiplication uses n^3 = 1,073,741,824 scalar multiplications. Ideal recursive Strassen at scalar leaves uses 7^10 = 282,475,249 scalar multiplications, around 3.80× fewer. This ideal count excludes additions, copying, allocation, padding and cache effects.

Why not always faster: seven recursive calls require extra additions/subtractions and temporary matrices; small matrices are dominated by overhead; memory traffic and cache locality matter; floating-point numerical error can grow (integer arithmetic avoids rounding but can still overflow fixed-width types). Practical hybrid: use Strassen only above a benchmarked cutoff and conventional multiplication below it; choose a power-of-two/padding strategy carefully.

(f) Final design and trade-offs

Module • Chosen algorithm • Time • Extra memory • Main trade-off • Change when

Orders • Stable Merge Sort; external merge if needed • Θ(n log n) • Θ(n) RAM or bounded buffers on disk • Extra storage for predictable time and stability • Data fits in RAM and an optimized stable library sort is preferable

Leaderboard • Randomized-pivot Quick Sort reference; production introsort • Expected Θ(n log n), worst bounded with introsort • Expected Θ(log n) stack • Randomization adds overhead; introsort is robust • Only a few scores change: maintain an index/ordered structure incrementally

Lookup • Lower-bound binary search • Θ(log n) • Θ(1) • Requires timestamp-sorted data • Data is unsorted or query volume is low enough that a scan is acceptable

Recommendations • Hybrid Strassen • Θ(n^2.807) asymptotically above cutoff • Additional temporary matrices; implementation-dependent • Fewer multiplications but more additions/memory traffic • Conventional/BLAS multiplication benchmarks faster at target size/hardware

Workload and implementation notes

- The 10 million order estimate assumes comparison sorting and log■ n; actual comparisons depend on implementation and data distribution.

- The 100,000-seller worst-case estimate illustrates the risk of deterministic first-element pivots, not a guaranteed runtime.

- Binary search maximum comparisons are logarithmic; this report uses 24 as a conservative bound for 10 million elements.

- Strassen's scalar multiplication comparison is an operation-count model; benchmark the actual implementation before production deployment.